

Desenvolvimento Teórico e Implementação Computacional da Síntese de Circuitos Lógicos Quânticos

Mapeamento do trabalho de Shende, Bullock e Markov (2006) para uma implementação computacional em Python

Síntese Teórica e Prática

30 de setembro de 2026

1 Introdução

A síntese de circuitos quânticos consiste em determinar uma sequência de portas quânticas elementares capaz de implementar uma transformação unitária previamente especificada. Em termos matemáticos, dado um operador unitário

$$U \in U(2^n),$$

o problema consiste em encontrar uma decomposição de U em portas pertencentes a uma biblioteca universal, normalmente formada por portas de um qubit e uma porta de dois qubits, como a CNOT.

O problema é particularmente relevante porque um operador genérico de n qubits é representado por uma matriz de dimensão

$$2^n \times 2^n.$$

Portanto, a dimensão do espaço de Hilbert cresce exponencialmente com o número de qubits. O artigo de Shende, Bullock e Markov destaca que essa estrutura exponencial se manifesta também na complexidade da síntese: operadores quânticos genéricos exigem, em geral, um número exponencial de portas de dois qubits.

A motivação para utilizar a quantidade de CNOTs como principal medida de custo está relacionada ao fato de que portas de dois qubits são consideravelmente mais difíceis de implementar fisicamente do que portas de um qubit. O artigo, portanto, utiliza a contagem de CNOTs como principal métrica de complexidade dos circuitos sintetizados.

A estratégia desenvolvida pelos autores baseia-se na introdução dos *Quantum Multiplexors* (QMUXs), que permitem formular uma espécie de análogo quântico da decomposição de Shannon utilizada em circuitos clássicos. A aplicação recursiva dessa estrutura conduz à chamada *Quantum Shannon Decomposition* (QSD).

O artigo estabelece quatro ideias centrais para a construção desse algoritmo:

1. Decomposição de operadores unitários por meio da *Cosine-Sine Decomposition* (CSD);
2. Interpretação dos fatores da CSD como multiplexadores quânticos;

3. Desmultiplexação de multiplexadores com cofatores unitários arbitrários;
4. Aplicação recursiva da decomposição até atingir operadores de dimensão suficientemente pequena.

O código desenvolvido neste trabalho reproduz computacionalmente essas etapas fundamentais. Em particular, as funções `qsd()`, `demux()` e `mux_rotation()` correspondem diretamente às estruturas matemáticas associadas, respectivamente, ao Teorema 13, Teorema 12 e Teorema 8 do artigo.

2 Formalismo Matemático

2.1 Qubits e espaço de Hilbert

Um qubit é representado por um vetor em um espaço vetorial complexo de dimensão dois:

$$\mathcal{H}_1 = \text{span}_{\mathbb{C}}\{|0\rangle, |1\rangle\}.$$

Os estados computacionais são representados por

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}.$$

Um estado arbitrário de um qubit pode ser escrito como

$$|\psi\rangle = \alpha_0 |0\rangle + \alpha_1 |1\rangle,$$

com

$$|\alpha_0|^2 + |\alpha_1|^2 = 1.$$

Para um registrador de n qubits, o espaço de Hilbert possui dimensão 2^n :

$$\mathcal{H}_n = \text{span}_{\mathbb{C}}\{|b\rangle : b \in \mathbb{B}^n\},$$

onde

$$\mathbb{B} = \{0, 1\}.$$

Consequentemente, um estado arbitrário é dado por

$$|\psi\rangle = \sum_{b \in \mathbb{B}^n} \alpha_b |b\rangle,$$

satisfazendo

$$\sum_{b \in \mathbb{B}^n} |\alpha_b|^2 = 1.$$

O crescimento exponencial da dimensão do espaço de Hilbert é um dos principais motivos para a necessidade de métodos estruturados de síntese.

2.2 Operadores unitários

A evolução de um sistema quântico fechado é representada por um operador unitário U , isto é,

$$U^\dagger U = U U^\dagger = I.$$

Para um registrador de n qubits,

$$U \in U(2^n).$$

A transformação do estado é dada por

$$|\psi\rangle \mapsto U |\psi\rangle.$$

A unitariedade garante que as normas e, portanto, as probabilidades de medição sejam preservadas.

No contexto de síntese, o objetivo é substituir a matriz U por uma sequência de operações elementares cuja composição seja exatamente equivalente a U , salvo erros numéricos de arredondamento.

2.3 Portas de um qubit

O artigo utiliza, entre outras, as rotações

$$R_y(\theta) = \begin{pmatrix} \cos(\theta/2) & \sin(\theta/2) \\ -\sin(\theta/2) & \cos(\theta/2) \end{pmatrix}$$

e

$$R_z(\theta) = \begin{pmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{pmatrix}.$$

Essas definições aparecem diretamente na implementação Python:

```
1 def Ry(phi):
2     c, s = np.cos(phi / 2), np.sin(phi / 2)
3     return np.array([[c, s], [-s, c]], dtype=complex)
4
5 def Rz(phi):
6     return np.diag([
7         np.exp(-1j * phi / 2),
8         np.exp(1j * phi / 2)
9     ])
```

Assim, a função `Ry` implementa exatamente a matriz matemática $R_y(\phi)$, enquanto `Rz` implementa $R_z(\phi)$.

Uma propriedade fundamental utilizada pelo artigo é a decomposição ZYZ . Qualquer operador unitário de um qubit pode ser escrito, a menos de uma fase global, como

$$U = e^{i\Phi} R_z(\alpha) R_y(\beta) R_z(\gamma).$$

Essa propriedade fornece a base para reduzir operadores arbitrários a rotações elementares.

3 Portas CNOT e custo de síntese

A porta CNOT é uma porta de dois qubits que utiliza um qubit de controle e um qubit alvo. Sua ação sobre estados da base é

$$|b_0 b_1\rangle \mapsto |b_0, b_0 \oplus b_1\rangle.$$

A CNOT é especialmente importante na síntese porque, combinada com portas de um qubit, constitui uma biblioteca universal.

No código, sua ação é construída explicitamente pela função `op_matrix`. Para cada estado da base, o programa identifica o bit de controle e, caso este seja igual a 1, inverte o bit correspondente ao alvo:

```
1 if bits[a]:
2     bits[b] ^= 1
```

A matriz global da CNOT é então construída para todo o espaço de Hilbert de n qubits.

A escolha da CNOT como métrica de custo é particularmente importante no trabalho de Shende et al., pois portas de dois qubits são consideradas mais dispendiosas e mais difíceis de implementar fisicamente que portas de um qubit.

4 Quantum Multiplexors

4.1 Definição

Um *Quantum Multiplexor*, ou QMUX, generaliza a ideia clássica de uma estrutura condicional `if-then-else`.

Considere um qubit de seleção e um registrador de dados. Um QMUX de um bit de seleção possui uma matriz na forma

$$U = U_0 \oplus U_1 = \begin{pmatrix} U_0 & 0 \\ 0 & U_1 \end{pmatrix}.$$

Quando o qubit de seleção está no estado $|0\rangle$, aplica-se U_0 ao registrador de dados; quando está em $|1\rangle$, aplica-se U_1 .

A estrutura permanece válida quando o qubit de seleção está em superposição. Por exemplo,

$$(\alpha |0\rangle + \beta |1\rangle) |\psi\rangle$$

é transformado em

$$\alpha |0\rangle U_0 |\psi\rangle + \beta |1\rangle U_1 |\psi\rangle.$$

Portanto, o QMUX não é simplesmente um “if” clássico. Ele preserva a coerência quântica entre os diferentes ramos da transformação.

O artigo introduz os QMUXs justamente porque essa estrutura permite organizar a síntese recursiva de operadores arbitrários.

4.2 Multiplexor Extension Property

Uma propriedade fundamental utilizada pelos autores é a *Multiplexor Extension Property* (MEP).

Se dois circuitos são equivalentes,

$$C \equiv D,$$

então é possível adicionar um mesmo qubit de controle às portas genéricas dos dois circuitos, preservando a equivalência:

$$C' \equiv D'.$$

Essa propriedade permite transformar decomposições de portas simples em decomposições multiplexadas. É justamente esse mecanismo que permite passar da decomposição *ZZZ* de uma porta de um qubit para a decomposição de rotações controladas por múltiplos qubits.

A MEP é, portanto, uma das pontes conceituais entre a decomposição de operadores individuais e a síntese de operadores de muitos qubits.

5 Rotações Multiplexadas

Uma rotação multiplexada consiste em aplicar uma rotação diferente ao qubit alvo para cada configuração dos qubits de controle.

Com k qubits de controle, existem

$$2^k$$

configurações possíveis. Assim, uma rotação multiplexada pode ser caracterizada por um vetor de ângulos

$$\boldsymbol{\theta} = (\theta_0, \theta_1, \dots, \theta_{2^k-1})^T.$$

O Teorema 8 do artigo mostra como decompor esse operador utilizando rotações de um qubit e CNOTs. O número final de CNOTs é

$$2^k.$$

O artigo destaca ainda que a decomposição pode ser organizada por meio de uma sequência de Código de Gray, permitindo que apenas um bit de controle mude entre etapas consecutivas.

5.1 Implementação computacional

A função responsável por essa etapa é:

```
1 def mux_rotation(ops, axis, target, controls, angles):
2     k = len(controls)
3     angles = np.asarray(angles, dtype=float)
4     assert len(angles) == 2 ** k
5
6     '''
7     if k == 0:
```

```

8     ops.append((axis, target, angles[0]))
9     return
10
11 gray = lambda i: i ^ (i >> 1)
12
13 M = np.array([
14     [(-1) ** bin(j & gray(i)).count("1")
15      for j in range(2 ** k)]
16     for i in range(2 ** k)
17 ])
18
19 phi = M @ angles / 2 ** k
20
21 for i in range(2 ** k):
22     ops.append((axis, target, phi[i]))
23
24     diff = gray(i) ^ gray((i + 1) % 2 ** k)
25     bit = diff.bit_length() - 1
26
27     ops.append(
28         ("CNOT", controls[k - 1 - bit], target)
29     )
30 '''

```

A primeira etapa da função determina o número de qubits de controle:

$$k = \text{len}(\text{controls}).$$

A condição

$$\text{len}(\text{angles}) = 2^k$$

é necessária porque cada configuração dos controles deve possuir um ângulo associado. Em seguida, é construído o Código de Gray por

$$g(i) = i \oplus (i \gg 1),$$

implementado em Python por

```

1 gray = lambda i: i ^ (i >> 1)

```

A função constrói também uma matriz de sinais M , relacionada à transformação de Walsh–Hadamard. Os ângulos físicos das rotações são obtidos por

$$\phi = \frac{1}{2^k} M \theta.$$

Essa transformação é necessária porque os ângulos θ_i descrevem o comportamento desejado para cada configuração dos controles, enquanto os ângulos ϕ_i correspondem às rotações individuais que aparecem efetivamente no circuito.

O código então alterna rotações de um qubit e portas CNOT. A variável `diff` identifica qual bit mudou entre dois elementos consecutivos do Código de Gray:

$$\text{diff} = g(i) \oplus g(i + 1).$$

O índice do bit alterado é obtido por

$$\text{bit} = \text{diff.bit_length}() - 1.$$

Esse mecanismo implementa diretamente a estrutura recursiva do Teorema 8.

6 Decomposição Cosseno-Seno

A CSD constitui o primeiro grande passo para a QSD.

Considere uma matriz unitária U de dimensão 2^n . Separando seus índices em dois grupos de tamanho 2^{n-1} , a CSD permite escrevê-la como

$$U = \begin{pmatrix} A_1 & 0 \\ 0 & B_1 \end{pmatrix} \begin{pmatrix} C & -S \\ S & C \end{pmatrix} \begin{pmatrix} A_2 & 0 \\ 0 & B_2 \end{pmatrix},$$

onde A_1 , A_2 , B_1 e B_2 são matrizes unitárias de dimensão 2^{n-1} e C e S são matrizes diagonais reais que satisfazem

$$C^2 + S^2 = I.$$

A estrutura central

$$\begin{pmatrix} C & -S \\ S & C \end{pmatrix}$$

possui a mesma estrutura matricial de uma rotação R_y . Entretanto, como C e S são diagonais, o ângulo da rotação pode depender da configuração dos demais qubits. Consequentemente, o bloco central pode ser interpretado como uma rotação R_y multiplexada.

Essa observação é essencial: a CSD transforma uma matriz unitária genérica em objetos que podem ser interpretados como QMUXs.

6.1 Implementação da CSD

No código, a CSD é realizada por

```
1 h = 2 ** (n - 1)
2
3 (u1, u2), theta, (v1h, v2h) =
4 cossin(U, p=h, q=h, separate=True)
```

A variável

$$h = 2^{n-1}$$

representa a dimensão dos blocos da matriz.

A função `cossin` da biblioteca `scipy.linalg` fornece os fatores correspondentes à decomposição CSD.

O resultado é dividido em três partes:

$$(u_1, u_2), \quad \theta, \quad (v_1^\dagger, v_2^\dagger),$$

que serão posteriormente convertidas em dois QMUXs genéricos e uma rotação R_y multiplexada.

7 Teorema 12: Desmultiplexação de um QMUX

O passo seguinte da construção da QSD é a desmultiplexação de um operador da forma

$$U_0 \oplus U_1.$$

O artigo procura matrizes unitárias V e W e uma matriz diagonal D tais que

$$U_0 \oplus U_1 = (I \otimes V)(D \oplus D^\dagger)(I \otimes W).$$

A partir dessa expressão, obtém-se a relação

$$U_0 U_1^\dagger = V D^2 V^\dagger.$$

Portanto, a matriz $U_0 U_1^\dagger$ pode ser diagonalizada para encontrar V e D .

Depois de determinar esses fatores, pode-se calcular

$$W = D V^\dagger U_1.$$

O termo central

$$D \oplus D^\dagger$$

é uma rotação R_z multiplexada, pois D é diagonal.

Essa construção constitui o Teorema 12 do artigo e é precisamente o mecanismo utilizado para substituir um QMUX genérico por dois operadores de dimensão menor e uma rotação multiplexada.

7.1 Implementação numérica por decomposição de Schur

A função correspondente no programa é:

```
1 def demux(ops, U0, U1, wires):
2     T, V = schur(
3     U0 @ U1.conj().T,
4     output="complex"
5     )
6
7     '''
8     D = np.exp(
9         0.5j * np.angle(np.diag(T))
10    )
11
12    W = np.diag(D) @ V.conj().T @ U1
13
14    rest = wires[1:]
15
16    qsd(ops, W, rest)
17
18    mux_rotation(
19        ops,
20        "RZ",
21        wires[0],
22        rest,
23        -2 * np.angle(D)
24    )
```

```

25
26 qsd(ops, V, rest)
27 '''

```

Primeiramente, o código calcula

$$U_0 U_1^\dagger$$

por

```

1 U0 @ U1.conj().T

```

A operação `conj().T` corresponde à transposta conjugada,

$$U_1^\dagger.$$

Em seguida, aplica-se a decomposição de Schur complexa:

$$U_0 U_1^\dagger = V T V^\dagger.$$

Para uma matriz unitária, T possui estrutura triangular unitária e seus elementos diagonais correspondem aos autovalores da matriz. O código extrai esses elementos por

```

1 np.diag(T)

```

e constrói

$$D = \text{diag}\left(e^{\frac{i}{2} \arg(\lambda_1)}, \dots, e^{\frac{i}{2} \arg(\lambda_m)}\right).$$

Dessa maneira,

$$D^2 = \text{diag}(\lambda_1, \dots, \lambda_m),$$

reproduzindo a estrutura necessária para

$$U_0 U_1^\dagger = V D^2 V^\dagger.$$

A escolha da decomposição de Schur é particularmente conveniente para a implementação numérica porque permite obter simultaneamente uma base unitária V e uma forma triangular cujos elementos diagonais contêm os autovalores.

Finalmente, calcula-se

$$W = D V^\dagger U_1,$$

exatamente como estabelecido pelo Teorema 12.

8 Teorema 13: Quantum Shannon Decomposition

A combinação da CSD com a desmultiplexação produz a QSD.

O Teorema 13 estabelece que um operador unitário arbitrário de n qubits pode ser decomposto em:

- quatro operadores genéricos de $(n - 1)$ qubits;
- três rotações multiplexadas.

Esse resultado é o ponto central do método, pois transforma o problema de sintetizar um operador de dimensão

$$2^n \times 2^n$$

em problemas menores de dimensão

$$2^{n-1} \times 2^{n-1}.$$

A recursão pode então ser aplicada aos quatro operadores menores.

8.1 Implementação da função qsd

A função principal do programa é:

```

1 def qsd(ops, U, wires):
2     n = len(wires)
3
4     '''
5     if n == 1:
6         ops.append(("U", wires[0], U))
7         return
8
9     h = 2 ** (n - 1)
10
11     (u1, u2), theta, (v1h, v2h) = \
12         cossin(U, p=h, q=h, separate=True)
13
14     demux(ops, v1h, v2h, wires)
15
16     mux_rotation(
17         ops,
18         "RY",
19         wires[0],
20         wires[1:],
21         -2 * theta
22     )
23
24     demux(ops, u1, u2, wires)
25     '''

```

A primeira condição é o caso-base:

```

1 if n == 1:
2     ops.append(("U", wires[0], U))
3     return

```

Quando resta apenas um qubit, não é necessário continuar a decomposição recursivamente. O operador unitário 2×2 é armazenado diretamente como uma porta genérica U .

Para $n > 1$, o programa calcula a CSD e, em seguida, realiza três operações principais:

1. desmultiplexa o bloco da direita;
2. implementa a rotação R_y multiplexada central;
3. desmultiplexa o bloco da esquerda.

Em termos esquemáticos,

$$U \longrightarrow \text{DEMUX} \rightarrow R_y^{\text{mux}} \rightarrow \text{DEMUX}.$$

Cada chamada a `demux()` gera duas chamadas recursivas a `qsd()`.

Portanto, cada nível da QSD produz quatro subproblemas de dimensão reduzida.

Essa é exatamente a origem da recorrência do número de CNOTs.

9 Recorrência do número de CNOTs

Seja c_n o número de CNOTs necessário para sintetizar genericamente um operador de n qubits.

A cada aplicação do Teorema 13 surgem quatro operadores de $n - 1$ qubits. Além disso, são necessárias três rotações multiplexadas.

Como uma rotação multiplexada com $n - 1$ qubits de seleção utiliza

$$2^{n-1}$$

CNOTs, obtém-se

$$c_n = 4c_{n-1} + 3 \times 2^{n-1}.$$

Para a implementação utilizada neste trabalho, a recursão termina em $n = 1$, para o qual

$$c_1 = 0.$$

Assim,

$$c_2 = 4c_1 + 3(2^1) = 6,$$

e

$$c_3 = 4c_2 + 3(2^2) = 4(6) + 12 = 36.$$

Portanto,

$$\boxed{c_3 = 36}.$$

O artigo apresenta exatamente essa recorrência e distingue esse resultado da versão otimizada que utiliza um caso-base de dois qubits e técnicas adicionais de absorção de diagonais.

10 Verificação Numérica

Uma característica importante da implementação é que ela não apenas gera uma sequência de portas, mas também reconstrói a matriz unitária produzida pelo circuito.

A função

```

1 def circuit_unitary(ops, n):
2     M = np.eye(2 ** n, dtype=complex)
3
4     '''
5     for op in ops:
6         M = op_matrix(op, n) @ M
7
8     return M
9     '''

```

inicia a evolução com a matriz identidade e multiplica sucessivamente as matrizes das portas.

Se as portas do circuito forem

$$G_1, G_2, \dots, G_m,$$

a matriz final é

$$U_{\text{circ}} = G_m G_{m-1} \cdots G_2 G_1.$$

Essa ordem é importante porque, como usual em álgebra linear, a primeira porta aplicada ao estado aparece mais à direita no produto matricial.

O programa calcula então o erro de reconstrução por meio da norma de Frobenius:

$$\epsilon = \|U_{\text{circ}} - U\|_F.$$

No código:

```

1 err = np.linalg.norm(
2     circuit_unitary(ops, n) - U
3 )

```

Finalmente, a implementação exige

```

1 assert count_cnots(ops) == 36 and err < 1e-9

```

Portanto, existem duas verificações independentes:

1. a contagem estrutural de CNOTs deve ser exatamente 36;
2. a matriz produzida pelo circuito deve coincidir numericamente com a matriz original.

Essa segunda condição é especialmente importante porque confirma que a decomposição numérica não apenas possui a quantidade esperada de portas, mas realmente implementa a transformação unitária especificada.

11 Geração da Unidade Aleatória

No exemplo principal, o programa considera

$$n = 3$$

e gera uma matriz unitária aleatória de dimensão

$$2^3 \times 2^3 = 8 \times 8.$$

A matriz é criada por

```

1 U = unitary_group.rvs(
2 2 ** n,
3 random_state=7
4 )

```

O parâmetro `random_state=7` torna o experimento reproduzível: execuções diferentes do mesmo programa utilizam a mesma realização pseudoaleatória.

O fluxo completo do programa pode ser resumido por

$$U_{8 \times 8} \xrightarrow{\text{CSD}} \{U_i, V_i, \theta\}$$

seguido por

$$\{U_i, V_i, \theta\} \xrightarrow{\text{demux}} \text{QMUXs menores} + R_z^{\text{mux}},$$

e finalmente

$$R_y^{\text{mux}}, R_z^{\text{mux}} \xrightarrow{\text{Teorema 8}} \text{rotações de 1 qubit} + \text{CNOTs}.$$

O resultado é uma sequência de operações elementares cuja matriz global reconstrói a unidade original.

12 Construção das Matrizes das Portas

em `ops` em uma matriz de dimensão $2^n \times 2^n$. Para portas de um qubit, o programa constrói a matriz global utilizando produtos de Kronecker:

$$A \otimes B.$$

Se uma porta G atua sobre o qubit a , os demais qubits recebem a identidade:

$$I \otimes \cdots \otimes I \otimes G \otimes I \otimes \cdots \otimes I.$$

O código realiza isso por:

```

1 mats = [np.eye(2)] * n
2 mats[a] = g
3
4 out = mats[0]
5
6 for m in mats[1:]:
7 out = np.kron(out, m)

```

Para a CNOT, a construção é diferente porque se trata de uma operação genuinamente de dois qubits.

O programa percorre todos os estados da base computacional, identifica o bit de controle e modifica o bit alvo quando necessário.

Essa abordagem possui uma interpretação direta:

$$|b_0 b_1 \cdots b_{n-1}\rangle \longrightarrow |b'_0 b'_1 \cdots b'_{n-1}\rangle,$$

onde

$$b_{\text{target}} \longrightarrow b_{\text{target}} \oplus b_{\text{control}}.$$

13 Fluxo Completo do Algoritmo

O algoritmo implementado pode ser organizado nas seguintes etapas.

13.1 Etapa 1 – Entrada

Recebe-se uma matriz unitária

$$U \in U(2^n).$$

No experimento,

$$U \in U(8).$$

13.2 Etapa 2 – CSD

A matriz é decomposta por

$$U = (A_1 \oplus B_1) \mathcal{R}_y (A_2 \oplus B_2),$$

onde $\mathcal{R}_y$ representa uma rotação multiplexada.

13.3 Etapa 3 – Desmultiplexação

Cada fator

$$A \oplus B$$

é transformado em

$$(I \otimes V)(D \oplus D^\dagger)(I \otimes W).$$

13.4 Etapa 4 – Conversão da parte diagonal

Como

$$D \oplus D^\dagger$$

possui estrutura diagonal compatível com uma rotação R_z , esse fator é implementado por uma rotação R_z multiplexada.

13.5 Etapa 5 – Síntese das rotações multiplexadas

Cada rotação multiplexada é transformada em rotações de um qubit e CNOTs por meio do Código de Gray e da transformação de Walsh–Hadamard.

13.6 Etapa 6 – Recursão

Os operadores V e W , que possuem dimensão correspondente a $n-1$ qubits, são novamente submetidos à QSD.

Esse processo continua até que

$$n = 1.$$

13.7 Etapa 7 – Reconstrução

A lista de operações produzida é convertida em uma matriz global U_{circ} .

13.8 Etapa 8 – Validação

Finalmente, verifica-se

$$\|U_{\text{circ}} - U\|_F < 10^{-9}$$

e

$$N_{\text{CNOT}} = 36.$$

14 Relação Direta entre o Artigo e o Código

A correspondência entre os principais resultados teóricos e suas implementações computacionais pode ser resumida na tabela a seguir.

Conceito	Artigo	Código
Rotação R_y	Seção II	<code>Ry()</code>
Rotação R_z	Seção II	<code>Rz()</code>
QMUX	Seção III	estrutura de <code>demux()</code>
MEP	Seção III	estrutura recursiva
Teorema 8	Rotações multiplexadas	<code>mux_rotation()</code>
CSD	Teorema 10	<code>cossin()</code>
Teorema 12	Desmultiplexação	<code>demux()</code>
Teorema 13	QSD	<code>qsd()</code>
CNOT	Porta elementar	<code>op_matrix()</code>
Produto do circuito	Composição unitária	<code>circuit_unitary()</code>
Contagem de CNOTs	Análise de custo	<code>count_cnots()</code>

A correspondência mais importante é:

$$\boxed{\text{qsd()} \longleftrightarrow \text{Teorema 13}}$$

$$\boxed{\text{demux()} \longleftrightarrow \text{Teorema 12}}$$

e

$$\boxed{\text{mux_rotation()} \longleftrightarrow \text{Teorema 8.}}$$

Assim, o código não constitui uma implementação independente da teoria: ele reproduz computacionalmente a sequência de decomposições introduzida pelos autores.

15 Arquiteturas com Vizinhos Mais Próximos

A contagem de CNOTs não é, por si só, suficiente para determinar a dificuldade física de implementação de um circuito.

Em várias arquiteturas quânticas, dois qubits só podem interagir diretamente quando são fisicamente adjacentes.

Considere uma cadeia

$$q_0 - q_1 - q_2 - \cdots - q_{n-1}.$$

Uma CNOT entre qubits não adjacentes precisa ser substituída por uma sequência de CNOTs entre vizinhos.

O artigo mostra que uma CNOT de comprimento k pode ser implementada utilizando

$$4k - 4$$

CNOTs de vizinhos mais próximos.

Além disso, para uma rotação multiplexada, existem CNOTs de diferentes comprimentos. A análise apresentada pelos autores leva ao limite

$$9 \times 2^{n-1} - 8$$

CNOTs de vizinhos mais próximos para a implementação da rotação multiplexada.

Consequentemente, a restrição de interação entre vizinhos pode aumentar a contagem de CNOTs por um fator de no máximo aproximadamente 9, sem alterar a ordem assintótica do algoritmo.

É importante ressaltar que essa etapa **não está implementada no código apresentado**. O programa produz CNOTs entre os índices especificados pelas variáveis `control` e `target`, sem introduzir automaticamente uma decomposição em CNOTs de vizinhos.

Portanto, essa seção representa uma extensão arquitetural da teoria do artigo, e não uma funcionalidade atualmente implementada no programa.

16 Complexidade Assintótica

A recorrência

$$c_n = 4c_{n-1} + 3 \times 2^{n-1}$$

mostra que a QSD possui crescimento exponencial.

O termo dominante da solução está associado ao fator

$$4^n.$$

Isso não é um artefato específico da implementação. A necessidade de crescimento exponencial está relacionada ao número de parâmetros independentes de um operador unitário genérico.

Para comparação, o artigo apresenta um limite inferior assintótico para operadores genéricos e mostra que a QSD otimizada se encontra a menos de um fator dois desse limite.

Essa característica é particularmente importante: embora a dimensão da matriz cresça como $2^n \times 2^n$, a síntese não cresce simplesmente como uma potência arbitrária maior, mas apresenta uma estrutura recursiva controlada por 4^n .

17 Limitações da Implementação

Apesar de reproduzir a estrutura fundamental da QSD, a implementação apresentada possui algumas simplificações em relação ao algoritmo completo discutido no artigo.

17.1 Ausência das otimizações do Apêndice

O artigo apresenta duas otimizações adicionais.

A primeira utiliza portas controladas- Z para implementar determinadas rotações multiplexadas R_y . Como essas portas são diagonais, uma delas pode ser absorvida pelo QMUX adjacente, reduzindo a quantidade total de CNOTs.

A segunda otimização explora decomposições especializadas de operadores de dois qubits. Os autores extraem fatores diagonais de operadores sucessivos e os combinam com multiplexadores adjacentes.

Essas otimizações são responsáveis pela redução adicional necessária para obter o limite final do artigo.

O código apresentado não implementa essas duas estratégias.

17.2 Caso-base

O código termina em

$$n = 1.$$

Consequentemente, operadores de dois qubits também são decompostos recursivamente.

Uma implementação mais próxima da versão otimizada do artigo poderia adotar

$$l = 2$$

como caso-base e utilizar uma decomposição especializada de operadores de dois qubits.

17.3 Ausência de otimização de portas

A lista `ops` representa as portas na ordem em que são produzidas, mas o código não realiza uma etapa geral de otimização posterior.

Por exemplo, ele não procura sistematicamente:

- CNOTs consecutivas que se cancelam;
- rotações consecutivas sobre o mesmo qubit;

- absorção de fatores diagonais;
- combinação de rotações;
- decomposições especiais para dois ou três qubits.

Assim, o circuito gerado deve ser entendido como uma implementação direta da decomposição matemática, e não como um circuito fisicamente otimizado.

18 Interpretação Conceitual

A principal contribuição conceitual da metodologia é transformar um problema global em uma sequência de problemas locais menores.

Inicialmente, temos

$$U_n.$$

A QSD produz quatro operadores

$$U_{n-1}^{(1)}, \quad U_{n-1}^{(2)}, \quad U_{n-1}^{(3)}, \quad U_{n-1}^{(4)}$$

e três rotações multiplexadas.

Cada operador U_{n-1} pode ser novamente decomposto:

$$U_{n-1} \longrightarrow 4U_{n-2} + 3R^{\text{mux}}.$$

Esse processo pode ser representado esquematicamente como

$$U_n \rightarrow 4U_{n-1} \rightarrow 16U_{n-2} \rightarrow 64U_{n-3} \rightarrow \dots$$

até alcançar o caso-base.

Essa estrutura é o análogo quântico da ideia de decomposição recursiva presente na síntese clássica de funções booleanas.

A grande diferença é que, no caso quântico, os “cofatores” são operadores unitários e os condicionamentos são realizados de maneira coerente por QMUXs.

19 Conclusão

O estudo desenvolvido demonstra a correspondência entre a teoria de síntese apresentada por Shende, Bullock e Markov e uma implementação computacional explícita da Decomposição Quântica de Shannon.

O processo inicia-se com uma matriz unitária arbitrária

$$U \in U(2^n)$$

e utiliza a CSD para dividi-la em blocos unitários menores e uma rotação multiplexada. Os blocos laterais são interpretados como QMUXs e desmultiplexados utilizando a relação

$$U_0 U_1^\dagger = V D^2 V^\dagger.$$

A matriz diagonal D dá origem a uma rotação R_z multiplexada, enquanto a CSD fornece a rotação R_y multiplexada central. As rotações multiplexadas são posteriormente

decompostas em rotações de um qubit e CNOTs por meio de uma transformação de Walsh–Hadamard organizada segundo o Código de Gray.

A função `qsd()` implementa a recursão principal do Teorema 13, enquanto `demux()` implementa a desmultiplexação do Teorema 12 e `mux_rotation()` realiza a síntese das rotações multiplexadas associadas ao Teorema 8.

Para três qubits, a escolha do caso-base de um qubit produz a recorrência

$$c_1 = 0,$$

$$c_2 = 6,$$

$$c_3 = 36.$$

O programa confirma esse resultado por meio da asserção

```
1 assert count_cnots(ops) == 36 and err < 1e-9
```

e, simultaneamente, verifica numericamente que

$$\|U_{\text{circ}} - U\|_F < 10^{-9}.$$

Dessa forma, a implementação fornece uma validação computacional da estrutura fundamental da QSD.

Entretanto, é necessário distinguir claramente essa implementação da versão completamente otimizada apresentada no artigo. O trabalho de Shende et al. emprega otimizações adicionais, incluindo a absorção de fatores diagonais e técnicas específicas para operadores de dois qubits, obtendo, no caso de três qubits, uma implementação com 20 CNOTs.

Assim, o resultado de 36 CNOTs obtido neste projeto não representa uma contradição com o resultado de 20 CNOTs do artigo. Ele corresponde à aplicação direta da recorrência da QSD com a recursão terminando em operadores de um qubit.

Por fim, o trabalho mostra como uma decomposição matemática de matrizes unitárias pode ser transformada em um procedimento computacional recursivo e verificável. A principal vantagem dessa abordagem está na estrutura modular: CSD, QMUX, desmultiplexação, rotações multiplexadas e CNOTs podem ser estudados individualmente e, posteriormente, combinados em um algoritmo completo de síntese quântica.

Além disso, a análise arquitetural apresentada no artigo mostra que a mesma estrutura pode ser adaptada a arquiteturas com interação apenas entre qubits vizinhos, mantendo a complexidade assintótica do método. Assim, a QSD fornece não apenas uma decomposição algébrica de operadores unitários, mas também uma estrutura conceitual para investigar a relação entre síntese, custo de portas e restrições físicas de implementação.

Referências

- [1] V. V. Shende, S. S. Bullock and I. L. Markov, “Synthesis of Quantum-Logic Circuits,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 25, no. 6, pp. 1000–1010, 2006. DOI: 10.1109/TCAD.2005.855930.